

Logout Does Not Terminate Session

Session invalidation handled in the browser only

Christian Bai | Redacted work sample

MEDIUM

This is a finding from a client penetration test, published as a writing sample. The client, the application, all hostnames, cookie names, tokens, routes and dates have been removed or replaced with placeholders. What remains is the structure of the write-up and the reasoning behind the severity.

Summary

Logging out does not terminate the session on the server. The application clears the session cookie from the browser, so the user appears to be signed out, but the token issued before logout stays valid server side and replayed requests continue to succeed.

The application sits behind an upstream identity provider, and the two layers behave differently at logout. The identity provider session ends correctly. The application's own token does not.

Impact

An attacker holding a captured session token can keep using it until it expires on the server, reaching any functionality the victim had access to, including the admin console, after the victim has logged out.

Logging out is the action a user takes when they believe they are ending their access. On a shared or public machine it is the only action available to them. The gap between what the browser shows and what the server honors is what makes this worth reporting at Medium rather than Low.

Evidence

All hosts, tokens, and cookie names below are placeholders. The sequence is unchanged.

1. Authenticated request before logout

```
GET /dashboard HTTP/2
Host: app.example.com
Cookie: SessionToken=[redacted]
```

```
HTTP/2 200 OK
```

2. Logout

```
GET /auth/logout HTTP/2
Host: app.example.com
Cookie: SessionToken=[redacted]

HTTP/2 302 Found
Location: https://idp.example.com/users/sign_out
Set-Cookie: SessionToken=; Path=/; Expires=Thu, 01 Jan 1970 00:00:00 GMT; HttpOnly;
Secure
```

The cookie is expired in the browser and the user is redirected to the identity provider to end that session too.

3. The identity provider session did end

```
GET /api/user_settings HTTP/2
Host: idp.example.com
Cookie: IdpSession=[redacted]

HTTP/2 401 Unauthorized
```

4. The application token did not

Replaying the pre-logout token against a protected application route:

```
GET /dashboard HTTP/2
Host: app.example.com
Cookie: SessionToken=[redacted]

HTTP/2 200 OK
```

The same token also retained access to the admin console:

```
GET /admin HTTP/2
Host: app.example.com
Cookie: SessionToken=[redacted]

HTTP/2 200 OK
```

5. Control

A request to the same route with no cookie redirected to `/sign-in`. This confirms the responses above were authenticated and were not public pages served to anyone who asks.

Recommendation

Invalidate the session server side when the user logs out instead of relying on cookie removal in the browser. The logout endpoint should revoke the token in the session store so that any later presentation of it fails authentication.

Terminating the upstream identity provider session is not sufficient on its own, because the application issues and accepts its own token independently of it.

Reference

OWASP Session Management Cheat Sheet